

SDLC Branch & Traceability Standards

Branch naming conventions, PR standards, and audit traceability

Engineering Standards | Version 1.0 | 2026

Why Traceability Matters

Audit Compliance

Every code change links back to a requirement, user story, or defect — providing a clear audit trail from request to deployment.

Change Impact Analysis

Trace which features, stories, and bugs are included in any release. Quickly identify what changed and why.

Accountability

Clear ownership of changes. Every branch, commit, and PR is tied to a trackable work item with an assigned developer.

Regulatory Requirements

SOC 2, ISO 27001, HIPAA, and SOX audits require demonstrable change management with full traceability.

Branch Protection Levels Defined

Two tiers of protection applied via GitHub branch protection rules

FULLY PROTECTED

Applies to: **main, develop**

- ✓ PR required for all changes — no direct pushes or commits
- ✓ Minimum 1 reviewer approval before merge
- ✓ All CI/CD status checks must pass
- ✓ Force pushes and branch deletion are disabled
- ✓ Signed commits recommended (optional enforcement)

STANDARD PROTECTED

Applies to: **release/*, hotfix/***

- ✓ PR required for all changes — no direct pushes
- ✓ Minimum 1 reviewer approval before merge
- ✓ CI/CD status checks must pass
- ✓ Force pushes disabled
- ✓ Branch deletion IS allowed (these are temporary branches)
- ✓ Relaxed: may allow maintainer override for time-sensitive releases

Mandatory vs Optional Branches

Not all long-lived branches are required — some depend on your team's Git workflow

REQUIRED

main

Production-ready code. Always exists.

Protection: Fully Protected

**feature/* | bugfix/* |
hotfix/***

Short-lived branches for individual work items.
Always branch from main or develop.
Always deleted after merge.

Protection: Not protected (ephemeral)

**Ticket ID in branch name is MANDATORY
for all feature/bugfix/hotfix branches.
This is the primary traceability link.**

OPTIONAL (if used, protect it)

develop

Integration branch for teams using Gitflow.
Feature branches merge here before release.
If your team uses trunk-based development
(feature branches merge directly to main),
you do not need this branch.
If it exists: Fully Protected

release/*

Stabilization branch created before a release.
Only needed if you separate release prep from
normal development. Temporary — deleted
after merging to main.

If it exists: Standard Protected

Rule: If a branch exists, it must be protected.

Branch Naming Convention

Standard Pattern

<type>/<ticket-id>-<short-description>

Optional — with PI and Sprint

<type>/pi<N>-s<N>/<ticket-id>-<short-description>

PREFIX

USE CASE

EXAMPLE (with optional pi/sprint)

feature/

New functionality tied to a user story

feature/IS-1234-add-user-profile

integration/

Multi-story feature (integration branch)

integration/IS-200-user-onboarding

bugfix/

Defect fix from backlog

bugfix/IS-567-fix-login-timeout

hotfix/

Urgent production fix (branches from main)

hotfix/IS-890-patch-auth-bypass

chore/

Tech debt, refactoring, CI/CD updates

chore/IS-321-upgrade-spring-boot

release/

Release stabilization branch

release/v2.4.0

Rules:

Lowercase only | Hyphens as separators | IS-xxxx ticket ID mandatory | PI/Sprint optional | Delete after merge

Pull Request Standards

PR Title Pattern

<type>(<scope>): <ticket-id> <description>

EXAMPLES

feat(auth): IS-1234 Add OAuth2 login flow

fix(api): IS-567 Resolve timeout on bulk export

hotfix(payments): IS-890 Patch null pointer in refund handler

chore(ci): IS-321 Upgrade to Gradle 8.12

PR REQUIREMENTS

- ✓ Link to ticket/user story in description
- ✓ At least 1 reviewer approval required
- ✓ All CI checks must pass
- ✓ No merge conflicts with target branch
- ✓ Squash-merge to keep history clean
- ✓ Delete source branch after merge
- ✓ Include test evidence or test plan
- ✓ Scope: one ticket per PR (preferred)

Commit Message Standard

Follow Conventional Commits (conventionalcommits.org)

Format

`<type>(<scope>): <short summary>`

`[optional body — explain WHY, not WHAT]`

`[optional footer]`

`Refs: IS-1234`

`BREAKING CHANGE: <description>`

ALLOWED TYPES

feat	- New feature	fix	- Bug fix	docs	- Documentat
style	- Formatting	refactor	- Restructure	test	- Add/fix tests
chore	- Maintenance	perf	- Performance	ci	- CI/CD changes

Ticket ID in commits: Optional (manual)

Ticket ID in branch name & PR title: Required

AUTOMATED TICKET INJECTION

Ticket ID in commits is OPTIONAL for developers.

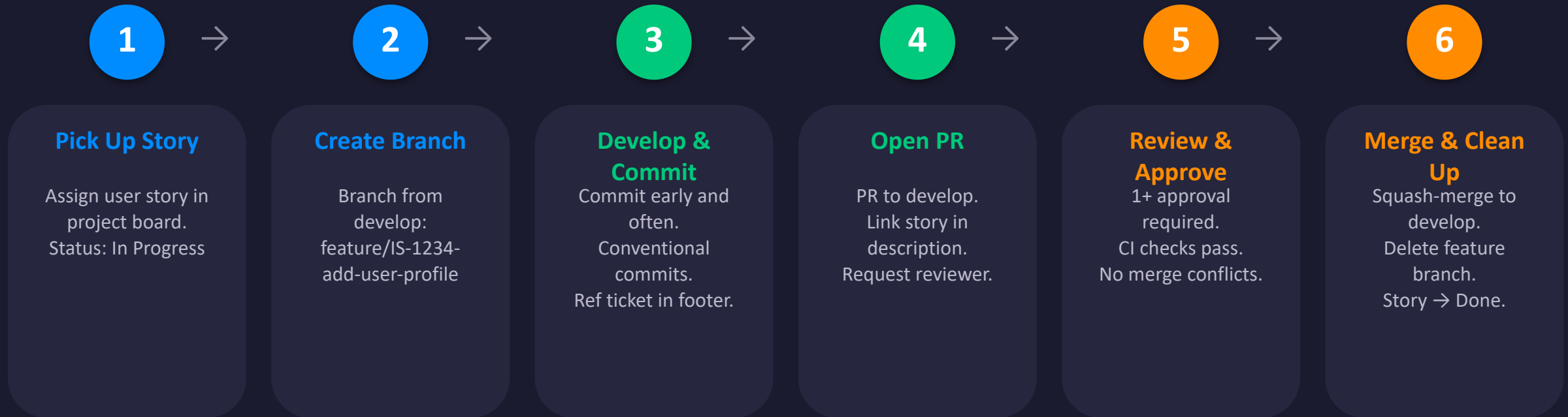
A Git hook (prepare-commit-msg) or CI step automatically extracts the ticket ID from the branch name and appends it to each commit.

How it works:

1. Dev creates branch: `feature/IS-1234-add-profile`
2. Dev commits: `feat(user): add profile page`
3. Hook detects IS-1234 from branch name
4. Commit becomes: `feat(user): add profile page`

`Refs: IS-1234 (auto-appended)`

Feature Branch Workflow



HOTFIX EXCEPTION

1. Branch from main (not develop): `hotfix/IS-890-patch-auth-bypass`
2. Fix, test, open PR to main with expedited review
3. After merging to main, immediately merge main back to develop
4. Hotfixes still require PR approval — no direct pushes to main

Work Item Hierarchy & Branch Mapping

Every feature has at least one user story — the ticket prefix in the branch name determines the pattern

YOUR WORK ITEM HIERARCHY

IS-100

Epic

No Branch

Strategic initiative or large body of work.
Planning container only — never branched.

IS-200

Feature

Conditional

Deliverable capability within an epic.
Always has 1 or more user stories.

IS-1234

User Story

Always Branch

Single piece of user-facing value.
The primary unit of work for developers.

IS-5678

Task

Rarely Branch

Technical sub-unit of a story.
Usually tracked within the story's branch.

TWO BRANCHING PATTERNS (both allowed)

The branch type prefix determines the pattern:

feature/

Pattern A — Single Story

Feature has 1 user story.
Branch at story level, PR to main/develop.

feature/IS-1234-signup-form → main

integration/

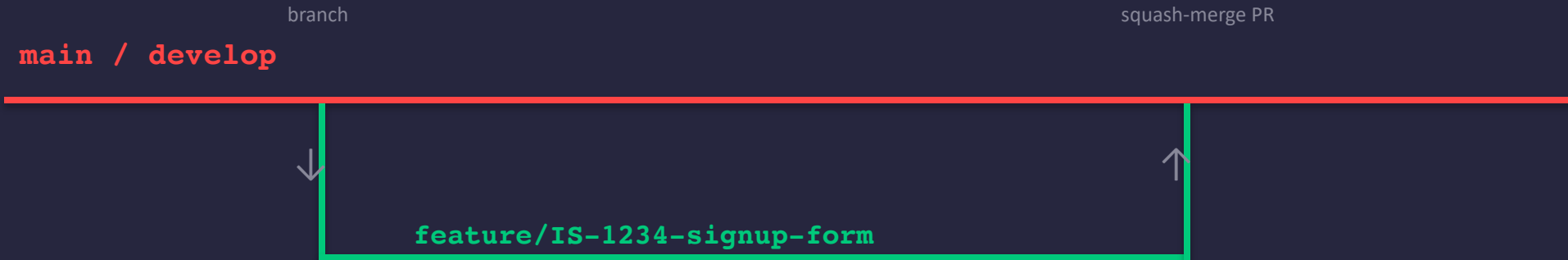
Pattern B — Multi-Story

Feature has 2+ user stories.
Integration branch at feature level.
Story branches merge into it.

integration/IS-200-onboarding → main
← feature/IS-1234-signup
← feature/IS-1235-verify

Pattern A: Single-Story Branching

Feature has one user story — branch directly at story level (most common pattern)



WHEN TO USE

- ✓ Feature maps to exactly 1 user story
- ✓ One developer (or pair) owns the full scope
- ✓ Work can be completed in a single PR
- ✓ Story is independently releasable

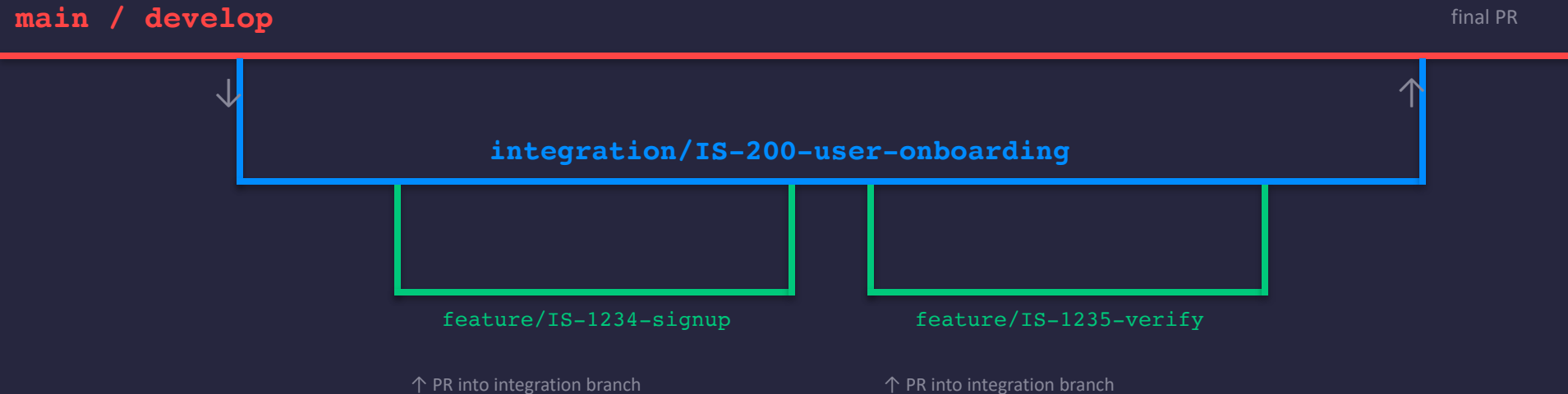
This is the default. Use this unless the feature has multiple stories.

LIFECYCLE

- 1. Pick up story** Assign IS-1234 on project board
- 2. Create branch** `feature/IS-1234-signup-form` from `main/develop`
- 3. Develop** Commit with conventional messages
- 4. Open PR** PR title: `feat(auth): IS-1234 Add signup form`
- 5. Review + merge** Squash-merge, delete branch, story → Done

Pattern B: Multi-Story Feature Branching

Feature has 2+ user stories — create a feature integration branch, story branches merge into it



RULES

- ✓ integration/ prefix triggers this pattern
- ✓ Story branches PR into integration branch
- ✓ Each story gets its own PR + review
- ✓ Integration branch PRs into main/develop
- ✓ Integration branch is Standard Protected
- ✓ Story branches deleted after merge

WHEN TO USE

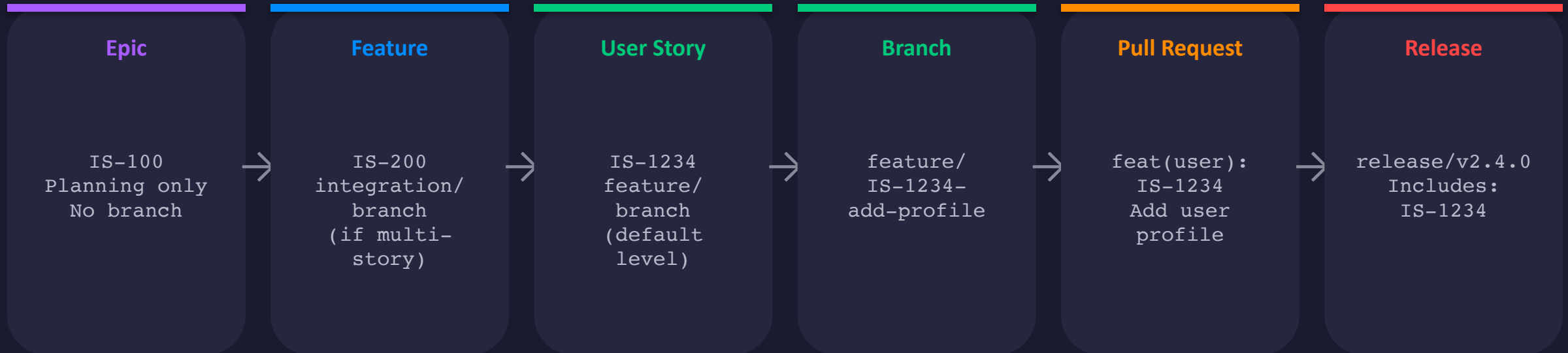
- ✓ Feature decomposes into 2 or more user stories
- ✓ Multiple developers working on the same feature
- ✓ Stories need to be integrated before release
- ✓ Feature needs to be tested as a whole before merging to main

WHY NOT JUST MERGE STORIES DIRECTLY?

Stories may depend on each other. The feature

End-to-End Traceability Chain

Every artifact links back to the originating requirement



AUDIT QUERIES THIS ENABLES

- ✓ "What code changed for IS-1234?" → Search branches & PRs by ticket ID
- ✓ "Who approved the change?" → PR approval history with reviewer names
- ✓ "Which release included this fix?" → Release branch contains merged PR
- ✓ "What was the business justification?" → PR links to user story with acceptance criteria

Commit Traceability Automation

Developers focus on code — tooling handles traceability

REQUIRED (enforced by CI)

- ✓ Ticket ID in branch name
- ✓ Ticket ID in PR title
- ✓ Conventional commit format
- ✓ PR linked to ticket in description

OPTIONAL (handled by automation)

- ✓ Ticket ID in commit message body/footer
- ✓ Refs: footer in each commit
- ✓ Cross-referencing related tickets
- ✓ Linking commits to deployment artifacts

AUTOMATION PIPELINE

prepare-commit-msg

LOCAL

Git Hook

Runs locally on each commit.
Extracts ticket ID from branch name and appends
Refs: <ticket-id> to the
commit message footer.



Branch Name Validator

CI

CI step runs on PR open.
Rejects PRs from branches
that don't match the pattern:
<type>/<TICKET-ID>--<desc>



PR Title Validator

CI

CI step checks PR title
matches conventional format
with ticket ID.
Blocks merge if invalid.



Merge Commit Tagger

CI

On squash-merge, the merge
commit automatically includes
the PR number and ticket ID
in the final commit message.

Quick Reference

BRANCH NAMING

`feature/IS-xxxx-short-desc`

`feature/pi3-s2/IS-xxxx-short-desc`

`integration/IS-xxxx-short-desc`

`bugfix/IS-xxxx-short-desc`

`hotfix/IS-xxxx-short-desc`

`chore/IS-xxxx-short-desc`

PR TITLE

`release/vX.Y.Z`

`feat(scope): IS-xxxx Description`

`fix(scope): IS-xxxx Description`

`hotfix(scope): IS-xxxx Description`

`chore(scope): IS-xxxx Description`

DO

- ✓ Include ticket ID in branch name
- ✓ One ticket per branch / PR
- ✓ Squash-merge feature branches
- ✓ Delete branch after merge

DON'T

- ✗ Push directly to main or develop
- ✗ Use vague branch names (my-fix, test-2)
- ✗ Merge without PR approval
- ✗ Leave stale branches after merge
- ✗ Skip ticket ID in branch or PR title

Enforcement & Tooling

GitHub Branch Protection

Enforce PR reviews, status checks, and prevent force pushes on main and develop.

`Settings → Branches → Add Rule`

Branch Name Lint

CI job validates branch name matches regex pattern. Blocks PR if non-compliant.

```
^(feature|integration|bugfix|hotfix|chore|release)/(p\d+-s\d+)?
```

PR Title Lint

GitHub Action or CI step validates PR title follows conventional commit format.

`commitlint` or `semantic-pr-title`

Commit Message Hook

pre-commit hook or CI check enforces conventional commits. Rejects non-compliant messages.

`commitlint + husky (local hooks)`

Automate enforcement wherever possible — convention without enforcement is just a suggestion.